FIFA 2026 World Cup P

Final Year Project Overview

Machine-learning system for international football outcome prediction, best-model selection, and large-scale World Cup tournament simulation.

Project Highlights

- 3-model comparison: Logistic Regression, Random Forest, XGBoost
- Time-based training and testing split
- Current best model selected using balanced accuracy
- Single-match prediction with probability outputs
- 48-team World Cup simulation over 10,000 iterations

Prepared from repository inspection, generated outputs, and current project artifacts

Prepared for final-year project overview and presentation support

FIFA 2026 World Cup Predictor

Master Project Overview

This document explains what the project is, why each major part exists, how the data and models flow through the system, what the repository produces, and what the current outputs show.

Current repository status inspected in this overview

- Historical training dataset with 12,124 international matches and 26 columns of pre-match and outcome information.
- Processed team-profile dataset with 170 latest team snapshots used for future prediction.
- Three trained models saved in the repository: Logistic Regression, Random Forest, and XGBoost.
- Best model currently selected: Logistic Regression, based on the project's fairness-aware selection rule.
- World Cup tournament simulation already generated over 10,000 iterations.

Contents

1. Executive summary
2. Project vision and problem being solved
3. End-to-end system architecture
4. Data foundation: raw dataset and processed team profiles
5. Feature engineering and target design
6. Model development and training strategy
7. Evaluation, comparison, and best-model selection
8. Match prediction and tournament simulation logic
9. Application layer and user experience
10. Repository structure and file responsibilities
11. Current outputs and what they mean
12. Limitations, strengths, and next-step improvements
13. Conclusion

Reader note. This overview is written to be presentation-ready: it explains both the technical design and the purpose of each component in plain, defendable language.

1. Executive summary

This final-year project is a machine-learning system for predicting international football match outcomes and using those predictions to simulate a full 48-team FIFA World Cup. The project learns from structured historical match data, compares three models, automatically selects the most suitable final model, predicts future matches through probabilities, and uses those probabilities inside a tournament simulator to estimate how often each team reaches every stage.

- Prediction target: Team A win, Draw, or Team B win.
- Core features: Elo ratings, recent-form statistics from the last five matches, match importance, tournament type, and neutral-ground context.
- Compared models: Logistic Regression, Random Forest, and XGBoost.
- Current final model: Logistic Regression, chosen because it achieved the best balanced accuracy and better draw handling than the other candidates.
- Deployment layer: a Streamlit app for single-match prediction and tournament-simulation viewing.

2. Project vision and problem being solved

Why this project matters. Football match prediction is difficult because outcomes are affected by team quality, recent form, match context, and tournament stakes. Rather than predicting exact scorelines directly, this project focuses on a more stable and practical target: the probability of each match outcome.

The project therefore solves two linked problems: (1) predicting the outcome probabilities of an international match before it is played, and (2) using those probabilities to estimate larger tournament outcomes such as qualification and championship chances.

Component	Why it exists
Historical dataset	Provides supervised examples so the model can learn outcome patterns from past international matches.
Team profiles	Stores the latest known Elo and form snapshot for each team, making future match prediction possible even when no new match row exists yet.
Three-model comparison	Prevents overcommitting to one algorithm and shows the project is evidence-driven.
Best-model selector	Automatically chooses the final deployable model using a fair, defendable rule.
Simulation engine	Turns match-level probabilities into tournament-level insight.
Streamlit app	Makes the project demonstrable, interactive, and presentation friendly.

3. End-to-end system architecture

The repository is organized as a modular pipeline. Data preparation, model training, evaluation, prediction, simulation, and presentation are split into separate scripts so the system remains easy to explain and easier to debug or extend.

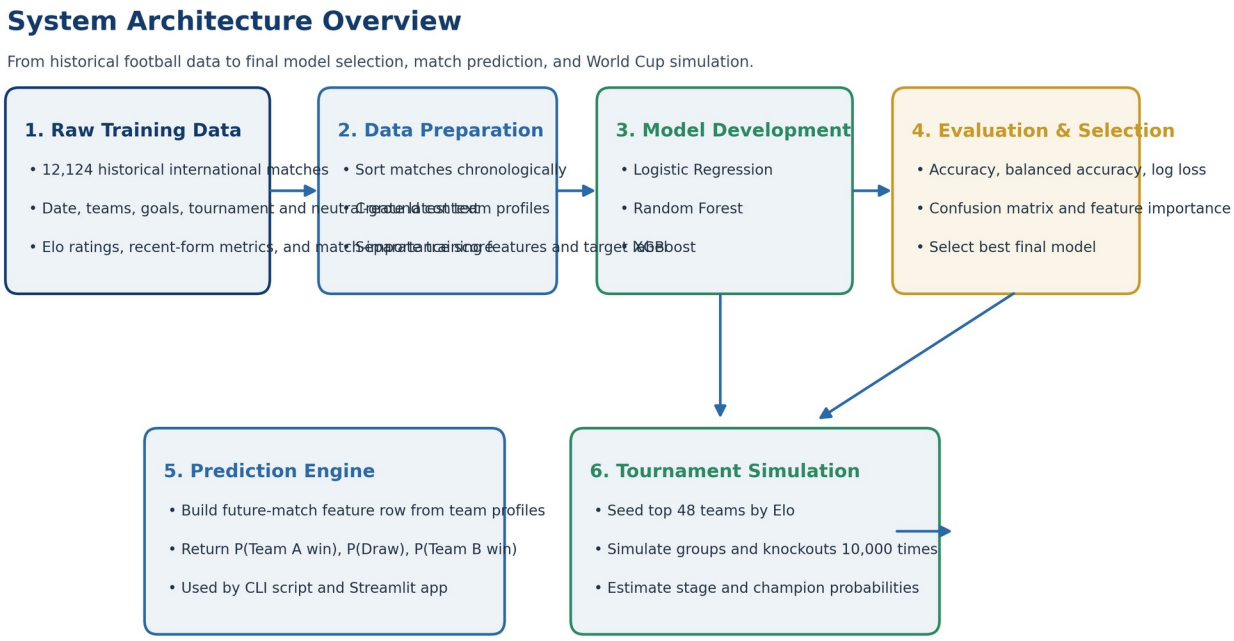

Figure 1. High-level architecture of the project from raw data to final simulation outputs.

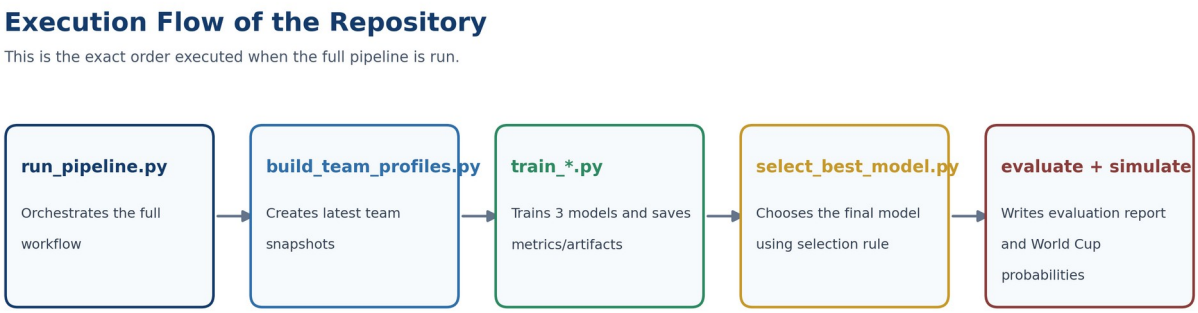

Figure 2. Exact execution order of the full repository pipeline.

4. Data foundation: raw dataset and processed team profiles

The project uses two very different but complementary data assets. Understanding the difference between them is one of the keys to understanding the whole FYP.

4.1 Raw training dataset

The raw file stores one row per historical international match. Each row contains the match date, the ordered teams (team_A and team_B), goals for each side, tournament and venue context, Elo ratings, last-five-match summaries for both teams, and the target label. This is the supervised dataset used to train the machine-learning models.

Raw dataset fact	Meaning
Rows	12,124
Columns	26

Date range represented in current files	2006-02-28 to 2026-01-25 (chronological split later isolates recent matches for testing)
Target column	label: 0 = Team A win, 1 = Draw, 2 = Team B win
Context columns	tournament, city, country, neutral, match_importance
Strength/form columns	elo_A, elo_B, elo_diff, last-five wins/draws/losses and goal averages for both teams

4.2 Processed team-profile dataset

The processed team-profile file is not a training set. It is a latest-state snapshot of each team. The script scans the historical data in chronological order and keeps updating every team's most recent Elo and recent-form values. At the end, each team has one row that can be used to build features for a future match.

Processed dataset fact	Meaning
Rows	170 teams
Columns	team, elo, wins_last5, draws_last5, loss_last5, gf_avg_last5, ga_avg_last5, gd_avg_last5, last_seen_date
Purpose	To generate pre-match features for future predictions and tournament simulation
Why it is necessary	A future match does not exist in the raw data yet, so the system needs a current snapshot for each team

4.3 What this distinction means in practice

- Raw dataset = historical examples used for learning.
- Processed team profiles = current team snapshots used for inference.
- The simulator and app do not retrain the model; they read the saved best model and combine it with the processed team profiles.

5. Feature engineering and target design

The model learns from structured, pre-match information. The feature design is intentionally football-specific but simple enough to stay explainable.

Feature group	Why it matters
Elo_A, Elo_B, Elo_diff	Represents overall team strength and the gap between both teams.
A/B wins, draws, losses in last 5	Captures short-term recent form and momentum.
A/B GF, GA, GD averages in last 5	Represents attack, defense, and net performance quality.
match_importance	Adds the competitive weight of the fixture.
tournament	Different competitions often have different difficulty and strategic behavior.
neutral	Accounts for neutral-ground conditions and reduced home advantage.

Target design

The project is a three-class classifier. The label is defined relative to the Team A / Team B ordering of the row: 0 means Team A wins, 1 means the match ends in a draw, and 2 means Team B wins.

This ordering matters because the feature row is directional: Team A and Team B have separate feature slots.

> **Presentation-ready explanation.** The system does not try to predict exact scores as its main task. It predicts the probability distribution over the three final match outcomes, which is more stable and directly useful for simulation.

6. Model development and training strategy

Three models are trained on the same feature set so the project can compare different learning behaviors on structured football data.

Model	Role in the project
Logistic Regression	Linear, fast, interpretable baseline that can still perform strongly on well-structured tabular features.
Random Forest	Ensemble of decision trees that can learn nonlinear relationships and feature interactions.
XGBoost	Boosted-tree model that is often strong on tabular classification problems.

Training pipeline design

- Chronological split: older matches are used for training and newer matches for testing. This avoids unrealistic leakage from future information.
- Preprocessing is embedded in a pipeline: numeric columns are median-imputed and standardized, while categorical columns are one-hot encoded.
- Each trained model is saved as a joblib artifact together with metadata such as feature lists, label map, and metrics.
- The project also builds a naive baseline for comparison, proving the trained models add real value beyond trivial class-frequency guessing.

7. Evaluation, comparison, and best-model selection

The project does not choose its final model based on one metric alone. Instead, it uses a fairness-aware rule that prioritizes balanced accuracy, then accuracy, then log loss. This is especially important in football because draws are usually the hardest class to predict and may be underrepresented by models that optimize only for overall accuracy.

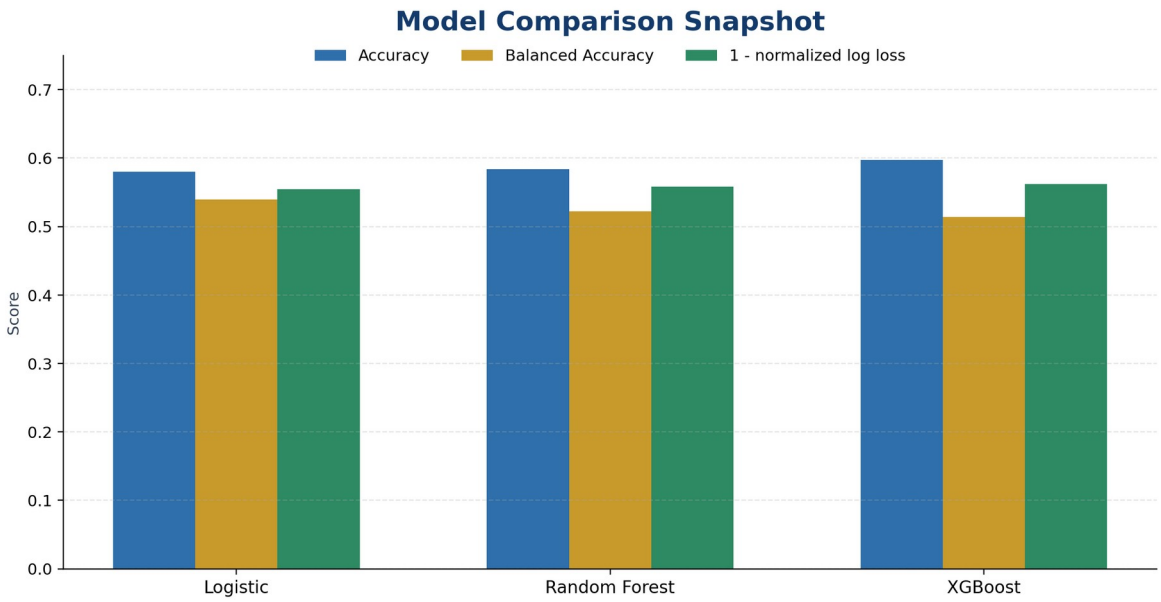

Figure 3. Comparison of the three candidate models using the saved project outputs.

Model	Accuracy	Balanced accuracy	Log loss	Training sec	Draw recall	Draw F1
logistic	0.580	0.540	0.891	0.20	0.277	0.292
random_forest	0.584	0.523	0.884	1.58	0.157	0.206
xgboost	0.598	0.514	0.876	0.75	0.058	0.098
naive_baseline	0.463	0.333	1.059	0.00	0.000	0.000

What the current results say

- Logistic Regression is the current final best model because it achieved the highest balanced accuracy (0.540) and the strongest draw handling among the trained models.
- XGBoost achieved the highest plain accuracy (0.598) and lowest log loss (0.876), but its draw recall is very low (0.058), which hurts balanced performance.
- Random Forest sits between both extremes: slightly better plain accuracy than Logistic Regression, but weaker draw-class handling.
- The selected best model therefore reflects the project's design goal: not just raw accuracy, but a more balanced classifier for three football outcomes.

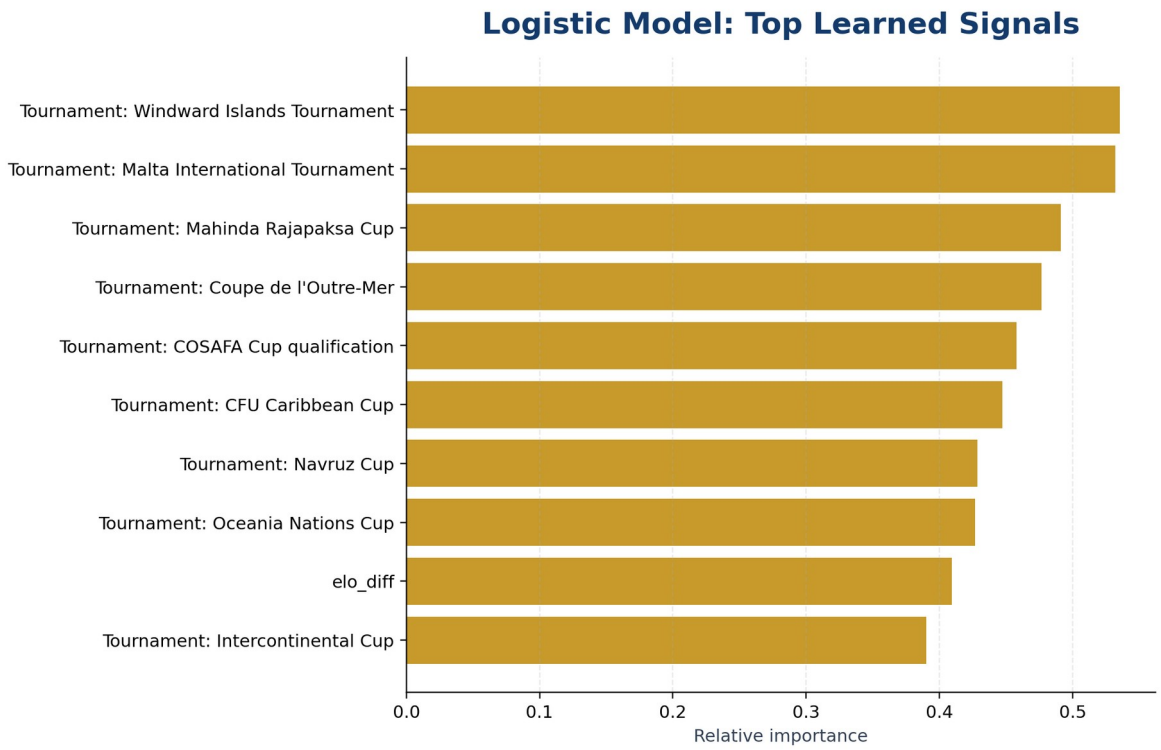

Figure 4. Snapshot of the strongest learned signals in the current logistic model.

8. Match prediction and tournament simulation logic

8.1 Single-match prediction

To predict a future match, the system loads the latest team profiles, builds one artificial pre-match row with Team A and Team B features, feeds that row to the saved best model, and returns three probabilities: Team A win, Draw, and Team B win. The highest probability becomes the displayed predicted outcome, but the full probability vector is preserved for deeper analysis and downstream simulation.

8.2 Tournament simulation

The tournament engine uses the best model as a probability generator. It first selects the top 48 teams by Elo, distributes them into 12 seeded groups, simulates every group match using sampled outcomes from the model's probabilities, advances the top two teams plus the best third-place teams, and then simulates knockout rounds until a champion is produced. This entire tournament is repeated 10,000 times to estimate stage probabilities.

Simulation stage	What happens
Step 1	Load best model and processed team profiles
Step 2	Seed top 48 teams by Elo into 12 groups
Step 3	Predict and sample every group-stage result
Step 4	Build standings using points, goal difference, goals for, and wins
Step 5	Advance 24 automatic qualifiers plus the best 8 third-place teams
Step 6	Simulate knockout rounds until a champion is produced
Step 7	Repeat the full tournament 10,000 times and aggregate stage frequencies

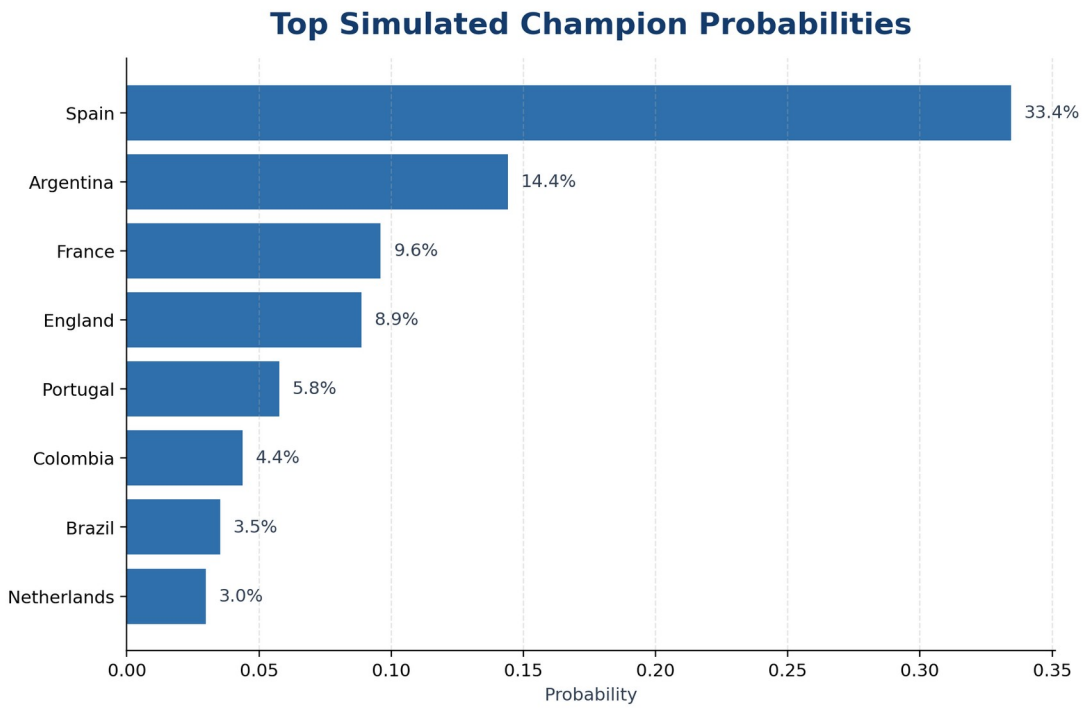

Figure 5. Highest simulated champion probabilities from the saved tournament outputs.

Team	Qualify	Quarterfinal	Semifinal	Final	Champion
Spain	97.9%	69.7%	55.9%	43.7%	33.4%
Argentina	94.0%	55.1%	38.5%	25.1%	14.4%
France	92.7%	48.5%	31.4%	18.5%	9.6%
England	89.9%	46.7%	29.8%	17.1%	8.9%
Portugal	88.8%	39.8%	23.4%	12.2%	5.8%

Current saved simulation outputs identify Spain as the strongest simulated favorite, followed by Argentina, France, England, and Portugal. These values are not official FIFA probabilities; they are the probabilities implied by this repository's feature design, best model, and simulation assumptions.

9. Application layer and user experience

The repository includes a Streamlit interface that makes the project easy to demonstrate. The app is split into two major pages.

App section	What the user sees
Match Prediction page	Lets the user choose Team A, Team B, neutral-ground setting, tournament name, and match-importance score; then displays the predicted outcome, probability distribution, and side-by-side team comparison.
Tournament Simulation page	Loads saved simulation results, shows stage-probability tables, lets the user sort top teams by different stages, and displays the seeded groups.

Why this matters in a defense or demo. The app proves the project is not only a training notebook; it is a complete pipeline with an accessible interface for prediction and scenario exploration.

10. Repository structure and file responsibilities

The codebase is intentionally modular. Each major directory has a clear role and each key script produces a specific output.

File	Responsibility
run_pipeline.py	Master orchestrator. Runs profile building, all model training, best-model selection, evaluation, and tournament simulation in sequence.
src/utils/helpers.py	Backbone utility file. Defines paths, feature lists, dataset loading, time-based split, team-profile generation, future-match feature building, artifact loading, and probability prediction.
src/data/build_team_profiles.py	Builds and saves the processed team_profiles.csv file from chronological raw data.
src/models/training_utils.py	Contains preprocessing, model definitions, training, metric computation, figure generation, feature-importance extraction, and naive baseline creation.
src/models/train_logistic.py	Wrapper script that trains and saves the Logistic Regression model.
src/models/train_random_forest.py	Wrapper script that trains and saves the Random Forest model.
src/models/train_xgboost.py	Wrapper script that trains and saves the XGBoost model.
src/models/select_best_model.py	Reads all metrics, compares models, applies the selection rule, copies the chosen artifact to best_model.joblib, and writes summary files.
src/evaluation/evaluate_model.py	Builds the final evaluation report for the selected best model.
src/predict/predict_match.py	Command-line entry point for predicting one future match.
src/simulation/tournament_simulation.py	Runs the 48-team World Cup Monte Carlo simulation and saves stage-probability outputs.
src/app/app.py	Streamlit application for interactive demonstration.
__init__.py files	Package markers that let the repository import modules cleanly across folders.

11. Current outputs and what they mean

Output artifact	Interpretation
models/*.joblib	Saved trained model artifacts for Logistic Regression, Random Forest, XGBoost, and the selected best model.
results/metrics_*.json	Per-model performance metrics and classification details.
results/model_comparison.csv	Machine-readable comparison table for all candidate models and the baseline.
results/best_model_summary.json	Final chosen model and the exact selection rule used.
results/evaluation_report.json	Summary report of the final best model.
results/top_features_*.csv	Ranked feature-importance outputs for each model.
results/worldcup_simulation_results.csv	Stage probabilities and champion probabilities from 10,000 tournament simulations.
results/auto_seeded_groups.csv	Automatically created group allocation used by the simulation.
reports/figures/*.png	Visual figures such as confusion matrices, feature-

	importance charts, and champion-probability plots.

12. Limitations, strengths, and next-step improvements

Major strengths

- Clear modular pipeline rather than a single monolithic notebook.
- Fair chronological train/test split that better matches real forecasting conditions.
- Model comparison across three suitable classifiers.
- Best-model selection that values balanced performance, not only headline accuracy.
- A complete prediction-to-simulation workflow with a usable front end.

Current limitations

- The project uses team-level pre-match features only; it does not include player injuries, lineups, bookmaker odds, or xG-style event data.
- Tournament qualification is approximated by top-48 Elo seeding, not by official FIFA qualification rules.
- Score generation in simulation is heuristic and not learned by a dedicated goals model.
- Team strength remains static during simulation; profiles are not updated after each simulated match.
- Some tournament one-hot features may be sparse because certain competitions appear less frequently in the historical data.

High-value future upgrades

- Add official qualification rules and a more realistic seeding mechanism.
- Update team strength dynamically during simulation to reflect momentum across the tournament.
- Introduce player-level or squad-level information for stronger pre-match context.
- Model scores separately using a dedicated goal model, then derive match outcomes from simulated scores.
- Calibrate probabilities further and evaluate reliability curves, not only classification metrics.

13. Conclusion

This FYP is a complete machine-learning pipeline for international football prediction and tournament simulation. Its value is not only that it produces a match winner forecast, but that it builds a coherent system: historical data becomes features, features become model probabilities, probabilities become tournament scenarios, and those scenarios become an interactive and explainable final product. The repository already demonstrates strong engineering structure, sensible model comparison, and presentation-ready outputs. With the current saved artifacts, the project's best deployable model is Logistic Regression and the simulation engine turns that model into a full World Cup prediction framework.

> **One-sentence defense summary.** This project predicts football match outcomes with machine learning and then scales those predictions into a full 2026 World Cup tournament simulator that estimates each team's chances of reaching every stage.